

TAREA 1: SIMULANDO ELOTELTAG Y APLICACIÓN FIND MY

NOMBRES: JULIÁN NÚÑEZ, VICENTE MUJICA, VALENTINA SOTO Y VICENTE ANDRADE
PROFESOR: AGUSTÍN GONZÁLEZ VALENZUELA

Introducción

La presente tarea tiene como objetivo simular el funcionamiento de un sistema de localización de objetos inspirado en la tecnología AirTag de Apple, denominado en este contexto EloTelTag.

El sistema modela un conjunto de dispositivos, entre ellos EloTelTags, celulares y tablets, que interactúan entre sí para reportar su posición geográfica a una nube central llamada EThube. Los EloTelTags y tablets, al no contar con GPS propio, se valen de celulares cercanos para transmitir su ubicación, mientras que los celulares reportan directamente su posición. A través de la aplicación FindMy, los usuarios pueden consultar en tiempo real la localización de todos sus equipos y pertenencias registradas.

El desarrollo de esta tarea se estructura en cuatro etapas incrementales. La primera etapa abarca la lectura de archivos de configuración y la creación de la clase EloTelTag. La segunda incorpora las clases Celular y EThube, habilitando el reporte de posiciones. La tercera añade la funcionalidad de visualización FindMy. Finalmente, la cuarta etapa integra la clase Tablet y completa todas las funcionalidades del sistema.

El programa recibe como entrada dos archivos: uno de configuración con los propietarios y sus dispositivos, y otro con los comandos de movimiento y consulta. Como salida, genera un archivo CSV con el historial de posiciones de cada equipo, el cual es posteriormente graficado para visualizar las trayectorias recorridas.

Para el desarrollo de esta tarea, se creó un repositorio en github <https://github.com/roadtothewest/EL0329-Tarea1>. Se clonó el repositorio del git en aragorn mediante ssh. Dentro del directorio, en la carpeta correspondiente a la etapa 4, al ejecutar en la CLI el comando `make run` se obtiene la siguiente warning:

Warning de compilación en Aragorn

```
make: atencion: Se ha detectado una desviacion en el reloj. La
      construccion podria estar incompleta.
```

La anterior se debe a una desincronización de reloj entre Aragorn y la máquina que realizó el push final. La ejecución de la aplicación no se ve afectada.

Stage 4

En esta etapa se agrega la clase `Tablet`, la cual extiende de la clase `Equipo`, y se modifican ciertos atributos y métodos asociados a la jerarquía de clases. Se incorpora además una lista de tablets en la clase `Territory`, de manera que ahora coexisten tres tipos de entidades: `Tags`, `Celulares` y `Tablets`. En este contexto, se implementan nuevos métodos como `getTablet` y `forEachTabletTryToReportLocation` dentro de `Territory`, con el objetivo de gestionar y consultar estos dispositivos de forma estructurada.

Adicionalmente, se agregan métodos destinados a establecer la configuración inicial de los equipamientos, tales como `setupTablet`. Por otra parte, se introduce lógica de control mediante estructuras `switch`, las cuales permiten diferenciar el comportamiento según el tipo de dispositivo, contemplando casos como `celular`, `tablet` y un caso por defecto (`EloTags`). Esta lógica se aplica tanto en la implementación del comando `findMy` como en los comandos destinados al desplazamiento de los equipos, permitiendo así un manejo más flexible y extensible del sistema.

Además, en la etapa 3 se crea una clase auxiliar denominada `Viewer`, la cual incorpora un método encargado de imprimir en consola los bienes asociados a una determinada persona. Esta clase permite separar la lógica de visualización del resto del sistema, facilitando la organización del código y mejorando su legibilidad.

Por último, el programa genera como salida un archivo en formato CSV, en el cual se puede observar la evolución de los equipos tras cada comando ejecutado. En este archivo es posible apreciar cómo los equipos que se encuentran dentro del radio de acción de los celulares reportan su posición a la nube. En la práctica, dichos equipos reportan la posición del celular más cercano, lo que provoca que en el archivo CSV se observe una aparente repetición de columnas, ya que múltiples equipos comparten las mismas coordenadas. Asimismo, cuando un equipo sale del radio de un celular, la nube conserva su última posición conocida; posteriormente, si el equipo vuelve a entrar en el radio de otro celular, su posición es actualizada conforme a la nueva referencia.

En resumen, la clase `SimuladorTest` se encarga de establecer la configuración inicial del sistema y de ejecutar la simulación. Por su parte, la clase `Territory` mantiene las listas de los distintos objetos presentes en el entorno, y proporciona métodos *getter* para obtener sus referencias, así como métodos que permiten reportar información hacia la nube.

La clase **ETNube** gestiona la actualización de sus listas internas a partir de la información recibida, e incorpora además métodos para imprimir los datos tanto en consola como en archivos CSV. En cuanto a la visualización, la clase **View** contiene el método responsable de imprimir en consola la información relevante, utilizando para ello datos provenientes de **ETNube** y de las instancias de **Equipo**.

Finalmente, la clase **Equipo** constituye la base de la jerarquía de dispositivos, a partir de la cual se derivan las clases **Celular**, **Tablet** y **EloTelTag**, compartiendo así atributos y comportamientos comunes.

Dada la anterior y el Makefile, ejecutamos `make` y `make run`, resultando en un archivo `output.csv`. Seleccionamos al usuario Pedro, dados los `move.txt` y `config.txt` utilizados en esta ejecución, ya que tiene 2 tag, una tablet y celular. Para 8 instantes, se obtienen las tablas 1 y 2.

En LibreOffice Calc se inserta un gráfico de dispersión donde se grafica la trayectoria de cada equipo de Pedro. Como se puede observar en la Figura 1, el gráfico refleja la lógica implementada en la simulación. Los dispositivos mochila y tablet se mantienen estáticos en sus coordenadas iniciales a lo largo de todos los instantes. Estos se encuentran fuera del rango de 10 metros de cualquier Celular y, por lo tanto, no pueden enviar actualizaciones a la nube.

La maleta de Pedro posee una coordenada inicial propia, su trayectoria se solapa con la del celular de Pedro. Cuando el celular experimenta un desplazamiento espacial en el paso 4, la maleta actualiza inmediatamente su posición en el sistema para reflejar las nuevas coordenadas del dispositivo móvil. Este fenómeno demuestra el cumplimiento del requerimiento principal: los ítems sin GPS actúan como balizas que reportan la ubicación del celular más cercano dentro de su radio.

Step	Pedro.celular.x	Pedro.celular.y	Pedro.maleta.x	Pedro.maleta.y
0	3,4	10,6	5	7,8
1	3,4	10,6	3,4	10,6
2	3,4	10,6	3,4	10,6
3	3,4	10,6	3,4	10,6
4	7,4	14,6	7,4	14,6
5	7,4	14,6	7,4	14,6
6	7,4	14,6	7,4	14,6
7	7,4	14,6	7,4	14,6

Table 1: Trayectorias (Step 0-7) - Celular y Maleta

Como se detalla en la Figura 2, la ejecución del sistema inicia en la clase **SimuladorTest**, la cual actúa como controlador principal instanciando los objetos centrales **Territory** y **ETNube**. Durante la lectura del archivo de configuración, el controlador crea instancias de **Celular**, **Tablet** y **EloTelTag**, todas derivadas de la superclase abstracta **Equipo**. Estos dispositivos son almacenados en las colecciones correspondientes de **Territory**, mientras que sus

Step	Pedro.mochila.x	Pedro.mochila.y	Pedro.tablet.x	Pedro.tablet.y
0	12,6	1,2	30,5	90,4
1	12,6	1,2	30,5	90,4
2	12,6	1,2	30,5	90,4
3	12,6	1,2	30,5	90,4
4	12,6	1,2	30,5	90,4
5	12,6	1,2	30,5	90,4
6	12,6	1,2	30,5	90,4
7	12,6	1,2	30,5	90,4

Table 2: Trayectorias (Step 0-7) - Mochila y Tablet

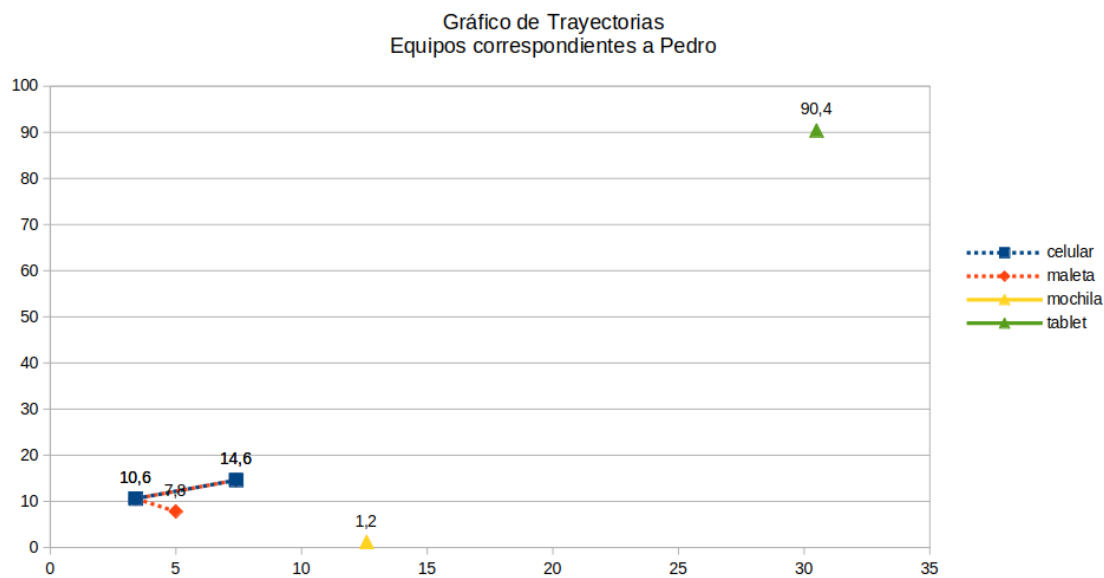


Figure 1: Gráfico de trayectorias de los equipos de Pedro.

coordenadas iniciales se registran de forma paralela en ETNube mediante la invocación del método `updateLocation()` de la instancia `ETNube`.

Para el procesamiento de `move.txt`, `SimuladorTest` recupera la referencia del dispositivo respectivo desde `Territory` e invoca su método heredado `move()`. Posteriormente, se gatilla el método `forEachTagTryToReportLocation()` de `Territory`, el cual itera sobre los dispositivos sin GPS y evalúa su distancia respecto a los celulares del entorno utilizando el método `isWithinRange()`. Si un celular se encuentra dentro del radio operativo, este asume el rol de transmisor e invoca `reportTagLocation()` o `reportTabLocation()` para actualizar las coordenadas del ítem en la nube.

Finalmente, la funcionalidad de consulta se activa cuando el sistema procesa un comando de visualización, invocando el método `comandoFindMy()` en el dispositivo emisor. En lugar de procesar la información de manera local, las clases `Cellular` y `Tablet` delegan la responsabilidad gráfica a la clase auxiliar `Viewer`. Esta última interactúa de manera directa con `ETNube` llamando al método `stringFormatoFigura2()`, encargado de extraer, clasificar y for-

matear el historial de posiciones almacenado para su despliegue estructurado en la consola.

Dificultades

En primer lugar, se presenta un problema al comparar cadenas de texto utilizando el operador `==` en lugar del método `equals`. Esta situación provoca que la comparación nunca resulte verdadera, debido a que en Java las cadenas corresponden a objetos y no a tipos primitivos, por lo que el operador `==` compara referencias en memoria en lugar de contenido. La resolución de este inconveniente se logra al emplear correctamente el método `equals`, el cual permite comparar el contenido de las cadenas.

En segundo lugar, la incorporación de la clase `Tablet` resulta un proceso tedioso, ya que existen múltiples funciones previamente definidas para la clase `EloTag` que también son necesarias para `Tablet`, tales como los métodos para reportar información a la nube o para determinar el celular más cercano. Si bien existen alternativas de diseño más adecuadas, la solución adoptada consiste en replicar y adaptar dichos métodos para la nueva clase.

Finalmente, se considera en algún momento la posibilidad de utilizar una clase que contenga como atributo una instancia de una clase derivada de sí misma, lo que introduce una relación circular. Esta situación genera dudas respecto a posibles fallos en la ejecución; sin embargo, el lenguaje Java permite este tipo de estructuras, siempre que se manejen adecuadamente para evitar problemas como referencias cíclicas no controladas.

Diagrama UML

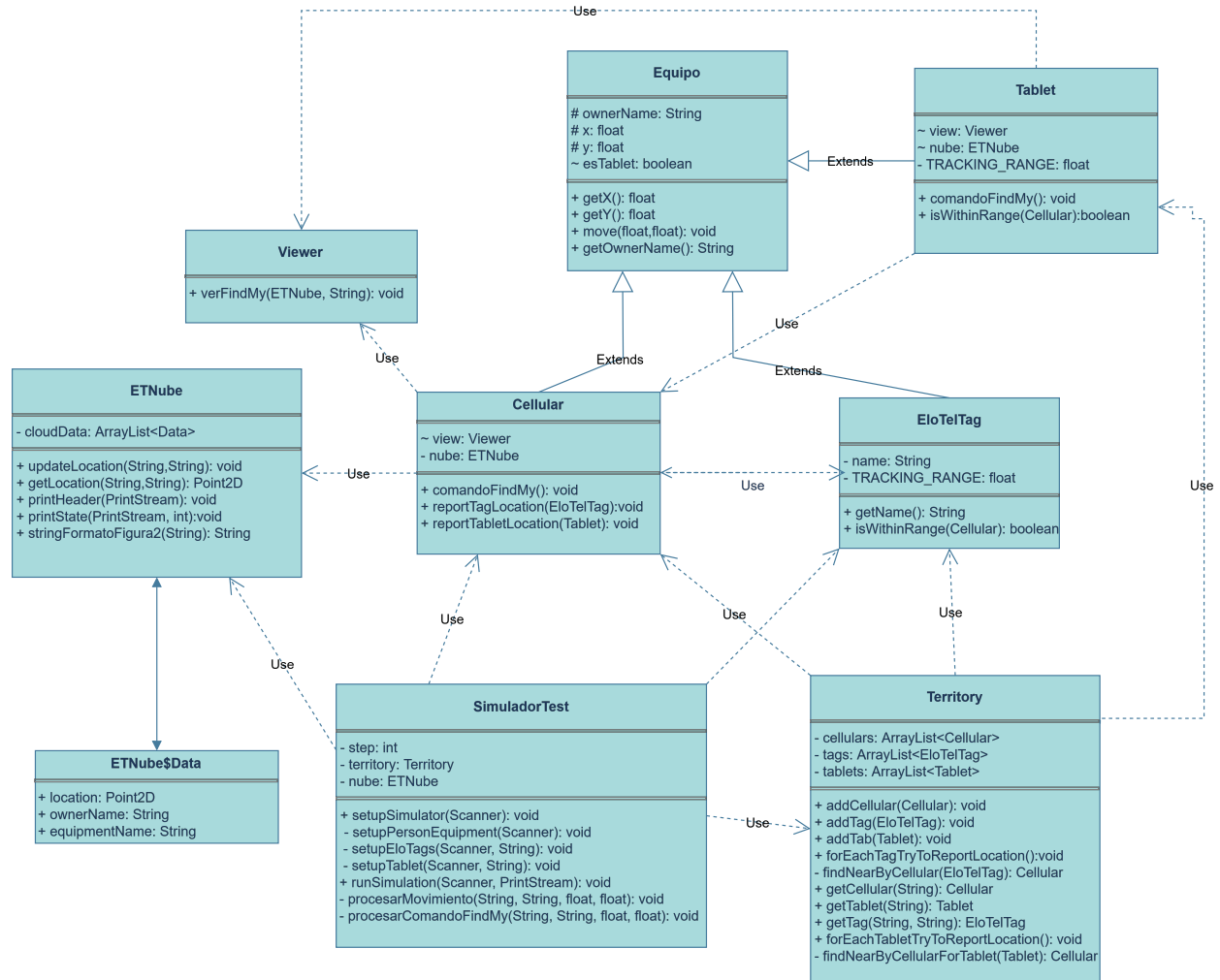


Figure 2: Diagrama de clases UML